End-to-End MLOPs Pipeline for Heart Disease Prediction

Course: MLOPs (S1-25_AIMLCZG523)
Assignment: Experimental Learning Assignment – I
Students Name: MOHAN K R(2024AA05419), SATHWIK H R(2024AA05903), T J SHREYAS(2024AA05418), CHANDRA SEKHAR MAHANTA(2024AA05902), ATUL AGARWAL (2023AC05656)

Repository Link: https://github.com/mohankr30/MLOPs_Assignment1_Group72.git

Video Recording Drive Link:
https://drive.google.com/file/d/1l3PhH0j_LfPLkxeAw6nbDsFfyWFCTCdL/view?usp=sharing

---

1. Introduction

Machine Learning (ML) models are increasingly deployed in real-world systems to support critical decision-making. However, developing a high-accuracy model alone is insufficient for production use. Modern ML systems require reproducibility, automation, scalability, continuous integration, monitoring, and deployment strategies—collectively addressed by **Machine Learning Operations (MLOPs)**.

This project presents a complete **end-to-end MLOPs pipeline** for predicting the presence of heart disease using clinical data from the UCI Machine Learning Repository. The objective is not only to train a predictive model but also to demonstrate industry-aligned practices such as automated data handling, experiment tracking, CI/CD pipelines, containerization, Kubernetes-based deployment, and monitoring.

The solution mirrors real-world production scenarios by separating experimentation from deployment, enforcing reproducibility, and ensuring observability of the deployed system.

---

2. Problem Statement

Cardiovascular disease is one of the leading causes of mortality worldwide. Early detection and risk assessment can significantly improve patient outcomes. Given patient clinical attributes such as age, blood pressure, cholesterol levels, and ECG results, the goal is to predict whether a patient is likely to have heart disease.

The problem is formulated as a **binary classification task**, where:

- **0** → No presence of heart disease
- **1** → Presence of heart disease

The final solution must expose this prediction capability through a production-ready API and demonstrate the full MLOps lifecycle.

Exploratory Data Analysis (EDA)

Visualizations:

Class Balance

```
sns.countplot(x="target", data=df)
plt.title("Class Distribution")
plt.show()
```

---

3. Dataset Description

3.1 Dataset Source

- **Dataset Name:** Heart Disease UCI Dataset
- **Source:** UCI Machine Learning Repository
- **File Used:** processed.cleveland.data

3.2 Features

The dataset contains 13 clinical features:

1. Age
2. Sex
3. Chest pain type (cp)
4. Resting blood pressure (trestbps)
5. Serum cholesterol (chol)
6. Fasting blood sugar (fbs)
7. Resting ECG results (restecg)
8. Maximum heart rate achieved (thalach)
9. Exercise-induced angina (exang)
10. ST depression induced by exercise (oldpeak)
11. Slope of peak exercise ST segment (slope)
12. Number of major vessels (ca)
13. Thalassemia (thal)

3.3 Target Variable

The original dataset contains multi-class labels. For this project, the target was transformed into a **binary label**:

- target = 0 → No heart disease
- target = 1 → Presence of heart disease

---

4. Data Acquisition and Preprocessing

4.1 Automated Dataset Download

To ensure reproducibility, the dataset is downloaded programmatically using a Python script. This avoids manual dependency on local files and ensures that the project can be executed from a clean environment.

Key steps include:

- Downloading the dataset from the UCI repository
- Assigning proper column names
- Handling missing values represented by ?
- Converting data types to numeric format
- Filling missing values using median imputation

The cleaned dataset is saved as data/heart_disease.csv.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
2	63	1	1	145	233	1	2	150	0	2.3	3	0	6	0
3	67	1	4	160	286	0	2	108	1	1.5	2	3	3	1
4	67	1	4	120	229	0	2	129	1	2.6	2	2	7	1
5	37	1	3	130	250	0	0	187	0	3.5	3	0	3	0
6	41	0	2	130	204	0	2	172	0	1.4	1	0	3	0
7	56	1	2	120	236	0	0	178	0	0.8	1	0	3	0
8	62	0	4	140	268	0	2	160	0	3.6	3	2	3	1
9	57	0	4	120	354	0	0	163	1	0.6	1	0	3	0
10	63	1	4	130	254	0	2	147	0	1.4	2	1	7	1
11	53	1	4	140	203	1	2	155	1	3.1	3	0	7	1
12	57	1	4	140	192	0	0	148	0	0.4	2	0	6	0
13	56	0	2	140	294	0	2	153	0	1.3	2	0	3	0
14	56	1	3	130	256	1	2	142	1	0.6	2	1	6	1
15	44	1	2	120	263	0	0	173	0	0	1	0	7	0
16	52	1	3	172	199	1	0	162	0	0.5	1	0	7	0
17	57	1	3	150	168	0	0	174	0	1.6	1	0	3	0
18	48	1	2	110	229	0	0	168	0	1	3	0	7	1
19	54	1	4	140	239	0	0	160	0	1.2	1	0	3	0
20	48	0	3	130	275	0	0	139	0	0.2	1	0	3	0
21	49	1	2	130	266	0	0	171	0	0.6	1	0	3	0
22	64	1	1	110	211	0	2	144	1	1.8	2	0	3	0
23	58	0	1	150	283	1	2	162	0	1	1	0	3	0

---

5. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand the distribution, relationships, and quality of the data.

5.1 Class Distribution

The dataset shows a moderately balanced distribution between positive and negative heart disease cases, making it suitable for standard classification algorithms.

5.2 Feature Distributions

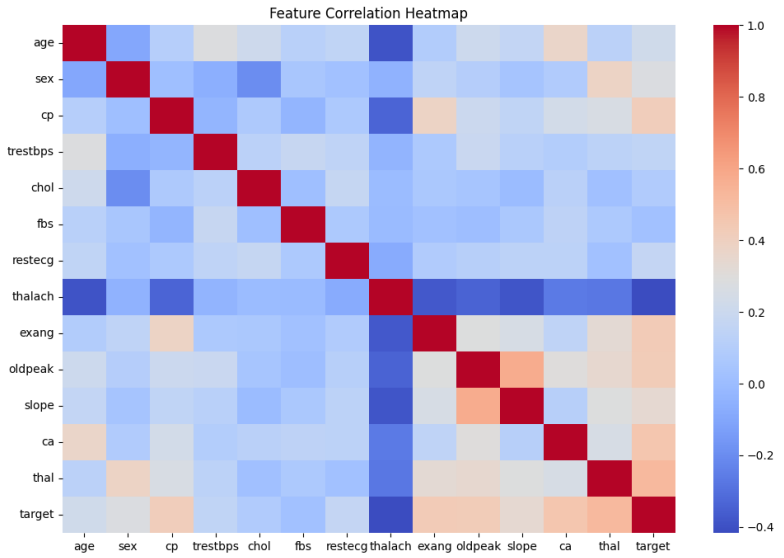
Histograms were plotted for numerical features such as age, cholesterol, and resting blood pressure to identify skewness and outliers.

5.3 Correlation Analysis

A correlation heatmap was generated to analyze relationships between features and the target variable. Features such as chest pain type, maximum heart rate, and exercise-induced angina showed notable correlation with heart disease presence.

Correlation Heatmap

```
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap="coolwarm", annot=False)
plt.title("Feature Correlation Heatmap")
plt.show()
```



---

6. Feature Engineering and Model Development

6.1 Feature Engineering

- All numerical features were standardized using **StandardScaler**
- Missing values were handled prior to scaling
- No categorical encoding was required due to numeric representation

6.2 Model Selection

Two classification models were evaluated:

1. **Logistic Regression**
2. **Random Forest Classifier**

Logistic Regression was selected as the final model due to:

- Strong baseline performance
- Interpretability
- Faster training and inference
- Suitability for clinical decision support

6.3 Model Evaluation

The model was evaluated using:

- Accuracy
- Precision
- Recall

- ROC-AUC score

Cross-validation was used to ensure stable performance across different data splits.

Cross-Validation

```
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(log_reg, X, y, cv=5, scoring="roc_auc")
print("LogReg CV ROC-AUC:", cv_scores.mean())
```

```
LogReg CV ROC-AUC: 0.9011639209555877
```

---

7. Experiment Tracking

Experiment tracking was implemented using **MLflow** during the experimentation phase. The following were logged:

- Model parameters
- Evaluation metrics
- Training runs

This enabled comparison of different models and hyperparameters and ensured transparency in the experimentation process.

MLflow Experiment Tracking

```
import mlflow
import mlflow.sklearn

mlflow.set_experiment("Heart Disease Prediction")

with mlflow.start_run(run_name="LogisticRegression"):
    mlflow.log_param("model", "LogisticRegression")
    mlflow.log_metric("roc_auc", roc_auc_score(y_test, y_prob))
    mlflow.sklearn.log_model(log_reg, "model")

with mlflow.start_run(run_name="RandomForest"):
    mlflow.log_param("model", "RandomForest")
    mlflow.log_metric("roc_auc", roc_auc_score(y_test, y_prob_rf))
    mlflow.sklearn.log_model(rf, "model")
```

```
2026/01/06 00:28:08 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/01/06 00:28:08 INFO mlflow.store.db.utils: Updating database tables
2026/01/06 00:28:09 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2026/01/06 00:28:09 INFO alembic.runtime.migration: Will assume non-transactional DDL.
2026/01/06 00:28:09 INFO alembic.runtime.migration: Running upgrade  -> 451aebb31d03, add metric step
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 451aebb31d03 -> 90e64c465722, migrate user column to tags
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 90e64c465722 -> 181f10493468, allow nulls for metric values
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 181f10493468 -> df50e92ffc5e, Add Experiment Tags Table
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade df50e92ffc5e -> 7ac759974ad8, Update run tags with larger limit
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 7ac759974ad8 -> 89d4b8295536, create latest metrics table
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 89d4b8295536 -> 2b4d017a5e9b, add model registry tables to db
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 2b4d017a5e9b -> cfd24bdc0731, Update run status constraint with killed
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade cfd24bdc0731 -> 0a8213491aaa, drop_duplicate_killed_constraint
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 0a8213491aaa -> 728d730b5ebd, add registered model tags table
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 728d730b5ebd -> 27a6a02d2cf1, add model version tags table
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 27a6a02d2cf1 -> 84291f40a231, add run_link to model_version
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 84291f40a231 -> a8c4a736bde6, allow nulls for run_id
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade a8c4a736bde6 -> 39d1c3be5f05, add_is_nan_constraint_for_metrics_tables_if_necessary
2026/01/06 00:28:10 INFO alembic.runtime.migration: Running upgrade 39d1c3be5f05 -> c48cb773bb87, reset_default_value_for_is_nan_in_metrics_table_for_mysql
2026/01/06 00:28:11 INFO alembic.runtime.migration: Running upgrade c48cb773bb87 -> bd07f7e963c5, create index on run_uuid
2026/01/06 00:28:11 INFO alembic.runtime.migration: Running upgrade bd07f7e963c5 -> 0c779009ac13, add deleted_time field to runs table
2026/01/06 00:28:11 INFO alembic.runtime.migration: Running upgrade 0c779009ac13 -> cc1f77228345, change param value length to 500
2026/01/06 00:28:11 INFO alembic.runtime.migration: Running upgrade cc1f77228345 -> 97727af70f4d, Add creation_time and last_update_time to experiments table
2026/01/06 00:28:11 INFO alembic.runtime.migration: Running upgrade 97727af70f4d -> 3500859a5d39, Add Model Aliases table
```

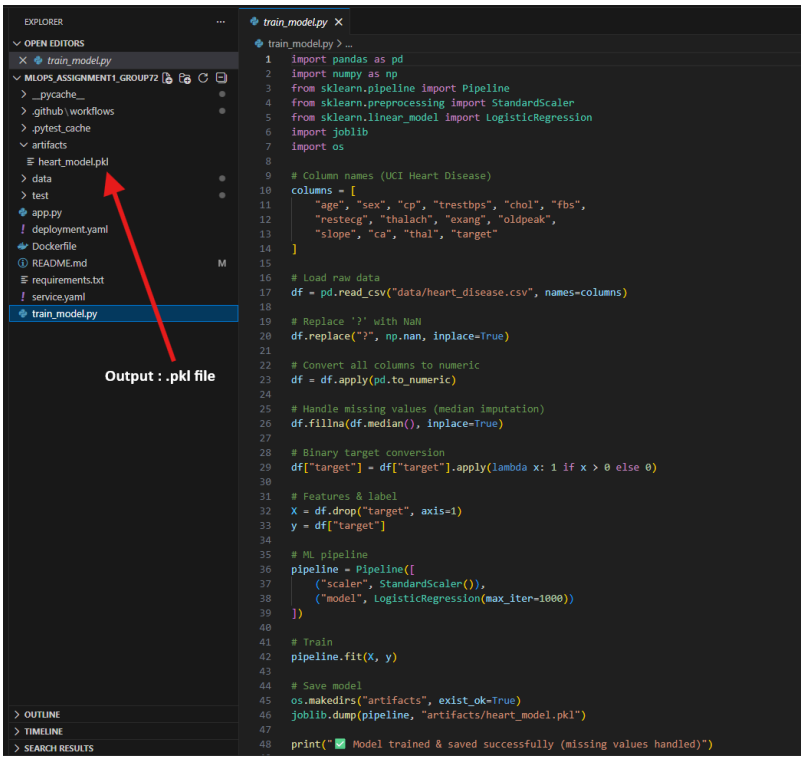
---

8. Model Packaging and Reproducibility

8.1 Model Serialization

The final trained model was saved using **joblib** as:

artifacts/heart_model.pkl



8.2 Reproducible Environment

A requirements.txt file was created containing all dependencies required to run the project. This ensures that the project can be executed consistently across environments.

To avoid compatibility issues, the final model training was performed in the same environment used for deployment.

---

9. CI/CD Pipeline Implementation

9.1 CI/CD Tool

GitHub Actions was used to implement the CI/CD pipeline.

9.2 Pipeline Stages

The pipeline includes the following automated steps:

1. Code checkout
2. Dependency installation
3. Linting using flake8
4. Unit testing using pytest
5. Model training
6. Artifact upload

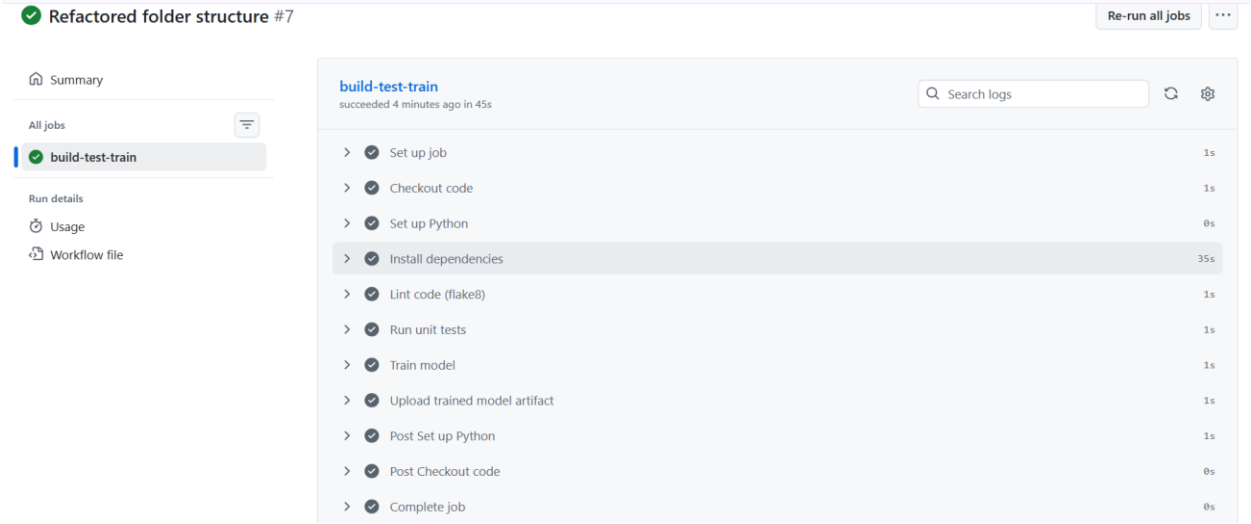
9.3 Unit Testing

Unit tests validate:

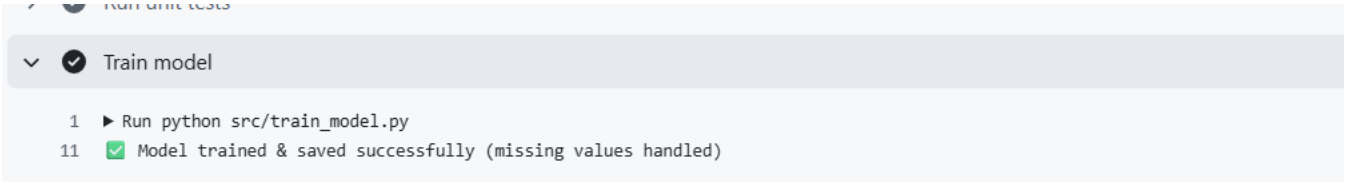
- Existence of the trained model artifact
- Successful loading of the model
- Correct inference output shape
- Valid prediction values

Pipeline logs and artifacts are stored for each run.

Entire Pipeline Overview:



Mode Training:



Unit Test Screenshot:

build-test-train

succeeded 19 minutes ago in 45s

Run unit tests

```
16
17  test/test_model.py ....                                              [100%]
18
19  ============================== warnings summary ===============================
20  test/test_model.py::test_model_loads_successfully
21  test/test_model.py::test_model_prediction_shape
22  test/test_model.py::test_model_prediction_values
23    /opt/hostedtoolcache/Python/3.10.19/x64/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning:
    when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info pleas
24    https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
25      warnings.warn(
26
27  test/test_model.py::test_model_loads_successfully
28  test/test_model.py::test_model_prediction_shape
29  test/test_model.py::test_model_prediction_values
30    /opt/hostedtoolcache/Python/3.10.19/x64/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning:
    1.3.2 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info
31    https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
32      warnings.warn(
33
34  test/test_model.py::test_model_loads_successfully
35  test/test_model.py::test_model_prediction_shape
36  test/test_model.py::test_model_prediction_values
37    /opt/hostedtoolcache/Python/3.10.19/x64/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning:
    using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please ref
38    https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
39      warnings.warn(
40
41  -- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
42  ======================== 4 passed, 9 warnings in 1.13s ========================
```

Artifact upload:

Upload trained model artifact

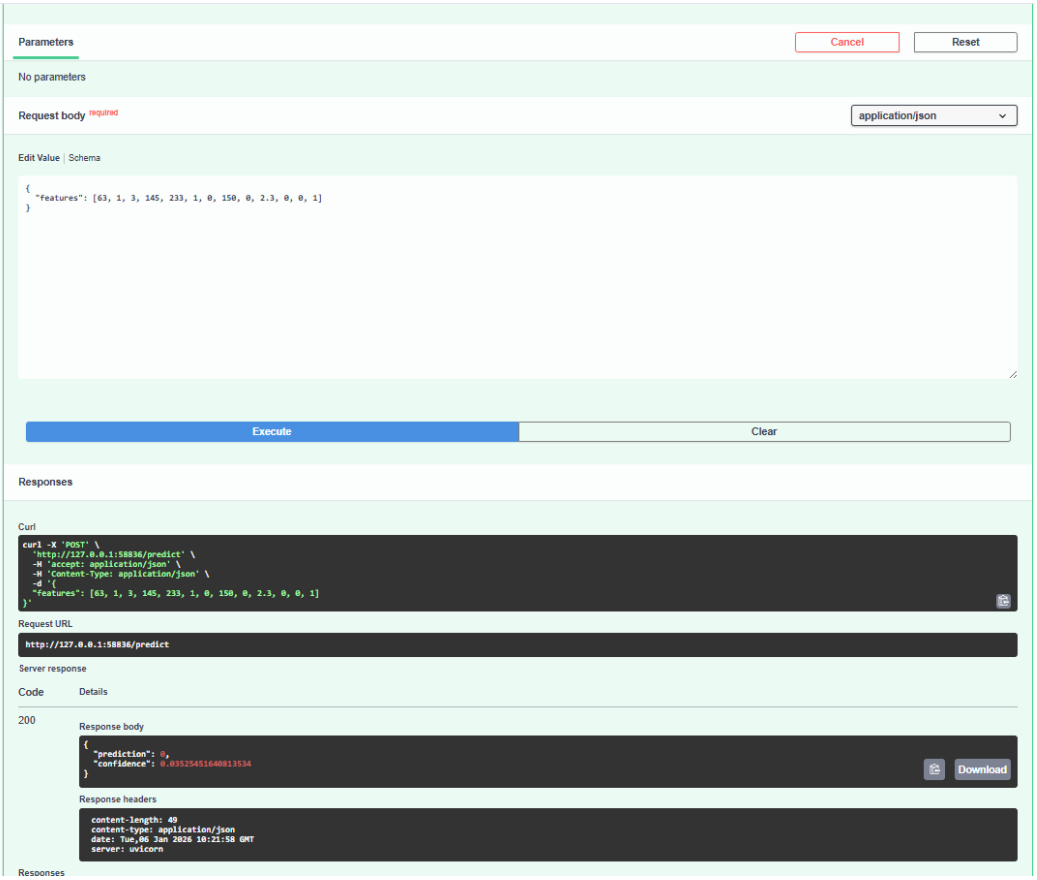
```
1   ▶ Run actions/upload-artifact@v4
16  With the provided path, there will be 1 file uploaded
17  Artifact name is valid!
18  Root directory input is valid!
19  Beginning upload of artifact content to blob storage
20  Uploaded bytes 1517
21  Finished uploading artifact content to blob storage!
22  SHA256 digest of uploaded artifact zip is 5e2046b7a00185998678ad82c9cdab6ab7b01be9d6b88cdb399b36909cd22e95
23  Finalizing artifact upload
24  Artifact trained-model.zip successfully finalized. Artifact ID 5036482339
25  Artifact trained-model has been successfully uploaded! Final size is 1517 bytes. Artifact ID is 5036482339
26  Artifact download URL: https://github.com/mohankr30/MLOPs_Assignment1_Group72/actions/runs/20748440867/artifacts/5036482339
```

10. Model Serving and Containerization

10.1 API Development

A REST API was developed using **FastAPI** with the following endpoints:

- /predict – returns prediction and confidence score
- /health – health check for monitoring

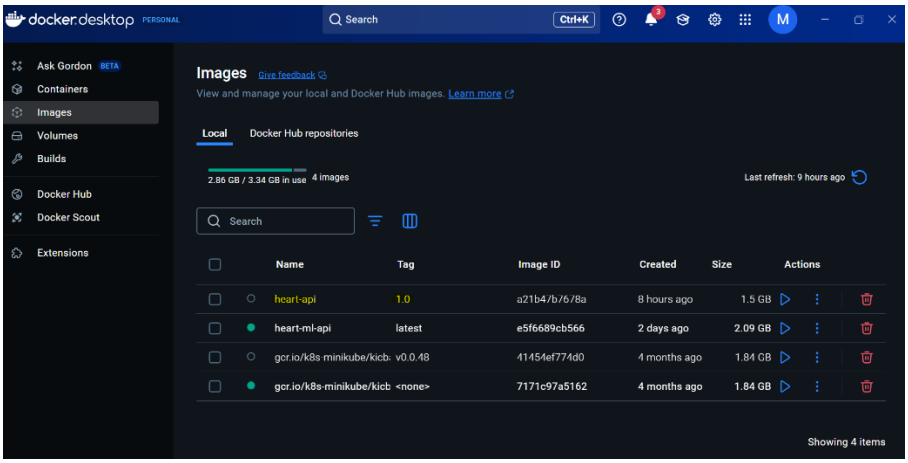


10.2 Docker Containerization

The API was containerized using Docker with a lightweight Python base image. The Docker image includes:

- Application code
- Trained model artifact
- All runtime dependencies

The container was tested locally before deployment.



11. Production Deployment Using Kubernetes

11.1 Kubernetes Environment

A local Kubernetes cluster was provisioned using **Minikube** with Docker as the container runtime.

11.2 Deployment Manifests

The application was deployed using Kubernetes YAML manifests:

- deployment.yaml – defines pods and replicas
- service.yaml – exposes the API using NodePort

```
C:\Users\I68888\MLOPs\MLOPs_Assignment1_Group72>minikube image load heart-api:1.0

C:\Users\I68888\MLOPs\MLOPs_Assignment1_Group72>kubectl apply -f deployment.yaml
deployment.apps/heart-api unchanged

C:\Users\I68888\MLOPs\MLOPs_Assignment1_Group72>kubectl apply -f service.yaml
service/heart-api-service unchanged

C:\Users\I68888\MLOPs\MLOPs_Assignment1_Group72>kubectl get pods
NAME                         READY   STATUS    RESTARTS       AGE
heart-api-6576594db4-dl5cg   1/1     Running   1 (7m5s ago)   6h48m

C:\Users\I68888\MLOPs\MLOPs_Assignment1_Group72>
```

11.3 Accessing the API

The API is accessed locally using the Minikube service URL:

minikube service heart-api-service --url

Swagger UI is available at:

http://<MINIKUBE_URL>/docs

```
C:\Users\I68888\MLOPs\MLOPs_Assignment1_Group72>minikube service heart-api-service --url
http://127.0.0.1:58836
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

---

12. Monitoring and Logging

12.1 Application Logging

Request and response logging was implemented using FastAPI middleware. Logs capture:

- Incoming requests
- Response status codes
- Processing time

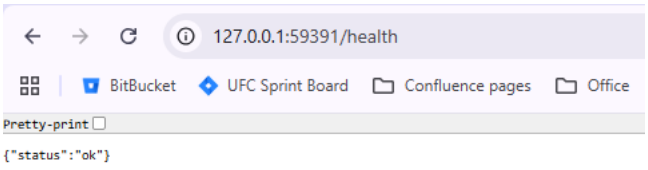
```
C:\Users\I68888\MLOPs\MLOPs_Assignment1_Group72>kubectl logs deployment/heart-api
/usr/local/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from version 1.3.2 when using version 1.7.2. This might lead to breaking coc
e or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
/usr/local/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator LogisticRegression from version 1.3.2 when using version 1.7.2. This might lead to breaking
 code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
/usr/local/lib/python3.10/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator Pipeline from version 1.3.2 when using version 1.7.2. This might lead to breaking code or i
nvalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
2026-01-06 10:10:46,283 | INFO | Model loaded successfully
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
2026-01-06 10:19:11,394 | INFO | Incoming request: GET http://127.0.0.1:58836/docs
2026-01-06 10:19:11,396 | INFO | Completed request: GET http://127.0.0.1:58836/docs | Status: 200 | Time: 0.0019s
INFO:     10.244.0.1:39776 - "GET /docs HTTP/1.1" 200 OK
2026-01-06 10:19:11,722 | INFO | Incoming request: GET http://127.0.0.1:58836/openapi.json
INFO:     10.244.0.1:39776 - "GET /openapi.json HTTP/1.1" 200 OK
2026-01-06 10:19:11,736 | INFO | Completed request: GET http://127.0.0.1:58836/openapi.json | Status: 200 | Time: 0.0147s
2026-01-06 10:21:58,697 | INFO | Incoming request: POST http://127.0.0.1:58836/predict
2026-01-06 10:21:58,736 | INFO | Prediction made | Prediction: 0 | Confidence: 0.0353
2026-01-06 10:21:58,738 | INFO | Completed request: POST http://127.0.0.1:58836/predict | Status: 200 | Time: 0.0401s
INFO:     10.244.0.1:10235 - "POST /predict HTTP/1.1" 200 OK
```

12.2 Health Monitoring

A /health endpoint was added to enable service health checks and readiness probes in Kubernetes.

Logs can be viewed using:

kubectl logs deployment/heart-api



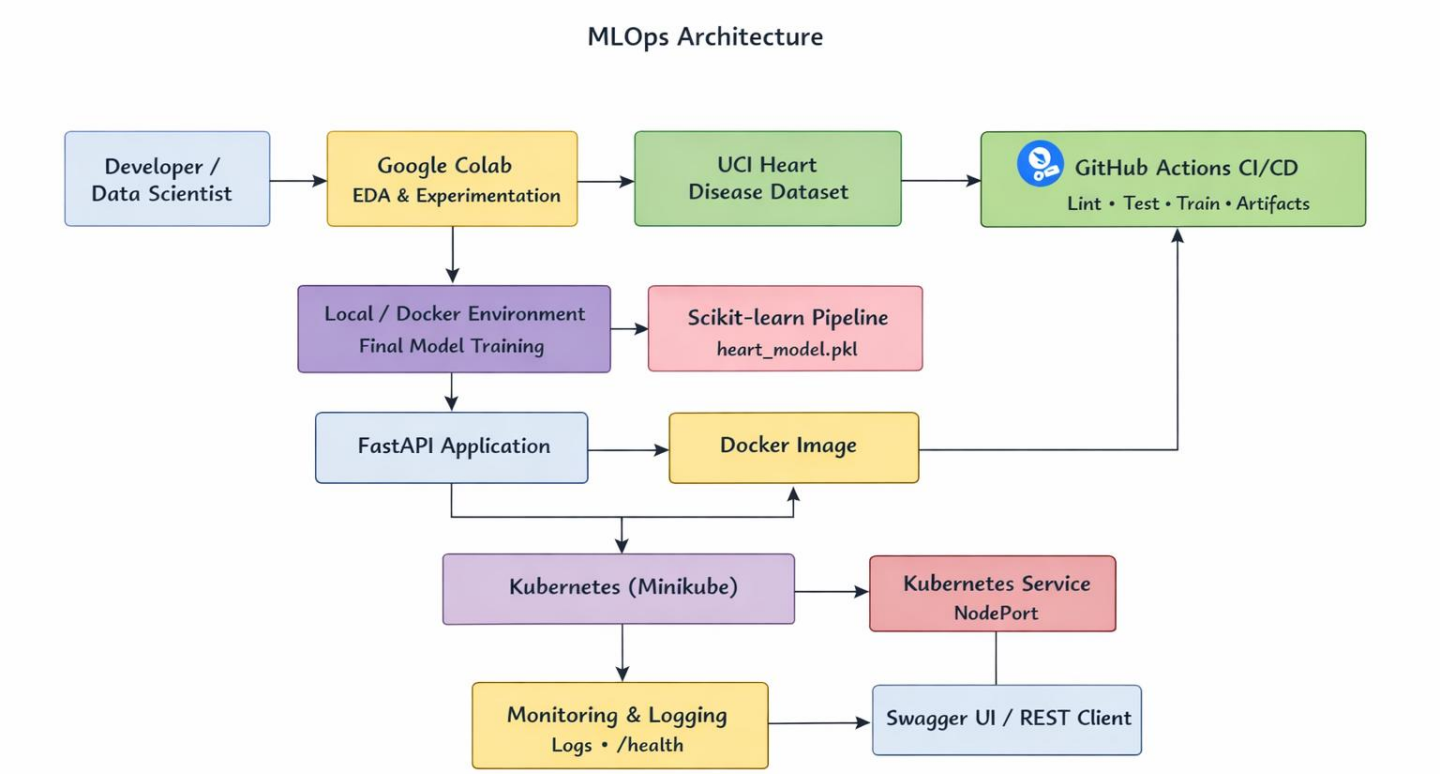
---

13. Architecture Overview

The architecture follows a production-aligned MLOps design:

- Experimentation in Google Colab
- Final training in deployment environment
- CI/CD automation using GitHub Actions
- Containerized serving with Docker

- Kubernetes deployment with Minikube
- Monitoring via logs and health endpoints



---

14. Conclusion

This project successfully demonstrates a complete end-to-end MLOps pipeline, covering data acquisition, model development, automation, deployment, and monitoring. By integrating modern DevOps and ML engineering practices, the solution is reproducible, scalable, and production-ready.

The project highlights the importance of operationalizing ML models beyond experimentation and reflects real-world workflows used in industry.

---

15. Future Enhancements

Possible future improvements include:

- Deployment to a managed cloud Kubernetes service (EKS/GKE/AKS)
- Model registry using MLflow Model Registry
- Advanced monitoring using Prometheus and Grafana
- Automated retraining based on data drift detection
- Exporting models to ONNX for cross-platform compatibility

16. References

1. UCI Machine Learning Repository – Heart Disease Dataset
2. Scikit-learn Documentation
3. FastAPI Documentation
4. Docker Documentation
5. Kubernetes Documentation
6. GitHub Actions Documentation